Arrays

One-dimensional arrays, strings, random numbers and 2D arrays (matrices)

Contents

Concepts	Why arrays?, Key rules, Finding the size with sizeof
Example 1	Array size with sizeof
Example 2	Reading elements and calculating the sum
Example 3	Largest element
Example 4	Counting even and odd numbers
Example 5	Reversing an array
Example 6	Shifting elements to the right
Example 7	Merging two arrays
Example 8	Adding two arrays element by element
Example 9	Decimal to binary
Example 10	Strings: character arrays
Example 11	Random numbers: different or repeated results
Example 12	Random numbers in a range [N, M] with functions
Example 13	Practice: 10 different two-digit random numbers
Example 14	2D array: printing a matrix
Example 15	Character matrix
Example 16	Matrix addition with functions
Example 17	Encrypting a character matrix (Caesar shift)

Why arrays?

Imagine storing seven numbers. Without arrays you need seven separate variables (`a, b, c, d, e, f, g`) and seven separate assignments. An array keeps all of them under one name, and a loop can process every element with a few lines of code.

```c
int numbers[7];            /* 7 ints: numbers[0] ... numbers[6] */

for (int i = 0; i < 7; i++) {
    numbers[i] = 0;        /* one line sets all 7 elements */
}
```

Key rules

- Indexes start at **0**. The last element of an array of size n is at index **n - 1**.
- An array can be initialized when it is declared: `int arr[] = {1, 2, 3};`. The size is counted automatically.
- C does not check the bounds. Reading or writing `arr[n]` is a bug that does not always crash.
- The size can come from user input (a variable-length array): `int values[n];` (C99 and later).
- A string is a `char` array that ends with the null character `'\0'`.

Finding the size with sizeof

`sizeof(array)` gives the total size in bytes and `sizeof(array[0])` gives the size of one element. Dividing them gives the number of elements:

```c
int arr[5] = {1, 2, 3, 4, 5};
int n = sizeof(arr) / sizeof(arr[0]);   /* 20 / 4 = 5 */
```

This only works in the function where the array is declared. Once an array is passed to a function, it becomes a pointer, so the size must be passed as a separate parameter.

EXAMPLE 1

Array size with sizeof

Print the total size of an int array, the size of one element and the number of elements.

CODE

```c
#include <stdio.h>

int main() {
    int arr[5] = {1, 2, 3, 4, 5};

    printf("Total size of the array: %zu bytes\n", sizeof(arr));
    printf("Size of one element: %zu bytes\n", sizeof(arr[0]));
    printf("Number of elements: %zu\n", sizeof(arr) / sizeof(arr[0]));
    return 0;
}
```

SAMPLE RUN

```
Total size of the array: 20 bytes
Size of one element: 4 bytes
Number of elements: 5
```

EXAMPLE 2

Reading elements and calculating the sum

Read 5 integers from the keyboard into an array and print their sum.

CODE

```c
#include <stdio.h>

int main() {
    int numbers[5];
    int i, sum = 0;             /* sum must start at 0 */

    for (i = 0; i < 5; i++) {
        printf("Enter element %d: ", i + 1);
        scanf("%d", &numbers[i]);
        sum += numbers[i];      /* sum = sum + numbers[i] */
    }
    printf("Sum of the numbers = %d\n", sum);
    return 0;
}
```

SAMPLE RUN

```
Enter element 1: 4
Enter element 2: 8
Enter element 3: 15
Enter element 4: 16
Enter element 5: 23
Sum of the numbers = 66
```

EXAMPLE 3

Largest element

Find the largest value in an array. Start by assuming the first element is the largest, then compare it with every other element.

CODE

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 8, 3, 7, 4};
    int max = arr[0];

    for (int i = 1; i < 5; i++) {
        if (arr[i] > max) {     /* 8 > 1: max = 8; 3 > 8? no; ... */
            max = arr[i];
        }
    }
    printf("Maximum element is: %d\n", max);
    return 0;
}
```

SAMPLE RUN

```
Maximum element is: 8
```

EXAMPLE 4

Counting even and odd numbers

Count how many elements of an array are even and how many are odd.

CODE

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5, 6};
    int evenCount = 0, oddCount = 0;

    for (int i = 0; i < 6; i++) {
        if (arr[i] % 2 == 0) {
            evenCount++;
        } else {
            oddCount++;
        }
    }
    printf("Even numbers: %d\n", evenCount);
    printf("Odd numbers: %d\n", oddCount);
    return 0;
}
```

SAMPLE RUN

```
Even numbers: 3
Odd numbers: 3
```

EXAMPLE 5

Reversing an array

Reverse the elements of an array in place by swapping the first element with the last, the second with the second-to-last, and so on.

CODE

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int n = sizeof(arr) / sizeof(arr[0]);
    int temp;

    for (int i = 0; i < n / 2; i++) {
        temp = arr[i];                  /* swap with a temporary variable */
        arr[i] = arr[n - i - 1];
        arr[n - i - 1] = temp;
    }

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
5 4 3 2 1
```

Note. If you only need to *print* the array backwards without changing it, a single loop is enough: `for (int i = n - 1; i >= 0; i--)`.

EXAMPLE 6

Shifting elements to the right

Shift every element one position to the right. The last element moves to the front.

CODE

```c
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int temp = arr[4];              /* save the last element */

    for (int i = 4; i > 0; i--) {
        arr[i] = arr[i - 1];
    }
    arr[0] = temp;

    printf("Array after shifting to the right: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
Array after shifting to the right: 5 1 2 3 4
```

EXAMPLE 7

Merging two arrays

Copy two 3-element arrays into one 6-element array.

CODE

```c
#include <stdio.h>

int main() {
    int arr1[] = {1, 2, 3};
    int arr2[] = {4, 5, 6};
    int merged[6];

    for (int i = 0; i < 3; i++) {
        merged[i] = arr1[i];
        merged[i + 3] = arr2[i];
    }

    for (int i = 0; i < 6; i++) {
        printf("%d ", merged[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
1 2 3 4 5 6
```

EXAMPLE 8

Adding two arrays element by element

Add two arrays of the same length and store the results in a new array.

CODE

```c
#include <stdio.h>

int main() {
    int arr1[] = {1, 2, 3};
    int arr2[] = {4, 5, 6};
    int n = sizeof(arr1) / sizeof(arr1[0]);
    int sum[n];

    for (int i = 0; i < n; i++) {
        sum[i] = arr1[i] + arr2[i];
    }

    printf("Sum array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", sum[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
Sum array: 5 7 9
```

EXAMPLE 9

Decimal to binary

Read a positive integer and print its binary form. Divide by 2 repeatedly, store each remainder in an array, then print the array backwards.

CODE

```c
#include <stdio.h>

int main() {
    int number, bits[32], index = 0;

    printf("Enter an integer: ");
    scanf("%d", &number);

    /* Part 1: collect the remainders (least significant bit first) */
    while (number > 0) {
        bits[index++] = number % 2;
        number /= 2;
    }

    /* Part 2: print them in reverse order */
    printf("Binary: ");
    for (int i = index - 1; i >= 0; i--) {
        printf("%d", bits[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN

```
Enter an integer: 13
Binary: 1101
```

Note. After the first loop, `index` equals the number of bits stored. For 13: 13%2=1, 6%2=0, 3%2=1, 1%2=1, so the array holds 1 0 1 1 and is printed as 1101.

EXAMPLE 10

Strings: character arrays

A string is stored as a char array that ends with `'\0'`. Find the length of a string by counting the characters before that marker.

CODE

```c
#include <stdio.h>

int main() {
    char name[6] = {'a', 'l', 'p', 'e', 'r', '\0'};  /* same as "alper" */
    char str[] = "Hello, World!";
    int length = 0;

    printf("name = %s\n", name);

    /* keep going until the null terminator is reached */
    while (str[length] != '\0') {
        length++;
    }
    printf("Length of \"%s\": %d\n", str, length);
    return 0;
}
```

SAMPLE RUN

```
name = alper
Length of "Hello, World!": 13
```

Note. To read a word use `scanf("%s", name);` with no `&`, because the array name is already an address. To read a whole line with spaces use `fgets(name, sizeof(name), stdin);`. The older `gets()` function was removed from the C standard because it can overflow the array.

EXAMPLE 11

Random numbers: different or repeated results

Fill a 10-element array with random numbers from 0 to 99. Write two versions: one that gives different results on every run, and one that gives the same results on every run.

CODE

```c
#include <stdio.h>
#include <stdlib.h>     /* rand, srand */
#include <time.h>       /* time */

int main() {
    int arr[10];

    srand(time(NULL));  /* version 1: seed from the clock -> different every run */
    /* srand(42); */    /* version 2: fixed seed          -> same every run      */

    for (int i = 0; i < 10; i++) {
        arr[i] = rand() % 100;      /* 0 ... 99 */
    }

    printf("Random array elements:\n");
    for (int i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN (random: your numbers will differ)

```
Random array elements:
13 65 78 78 40 66 21 20 20 73
```

EXAMPLE 12

Random numbers in a range [N, M] with functions

Read X, N and M. Generate X random numbers between N and M (inclusive), store them in an array, and print them. Generating and listing are done in separate functions.

CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void generateNumbers(int numbers[], int N, int M, int X) {
    for (int i = 0; i < X; i++) {
        numbers[i] = N + rand() % (M - N + 1);   /* N ... M */
    }
}

void listNumbers(int numbers[], int X) {
    for (int i = 0; i < X; i++) {
        printf("%d\n", numbers[i]);
    }
}

int main() {
    int X, N, M;
    srand(time(NULL));

    printf("How many random numbers? ");
    scanf("%d", &X);
    printf("Minimum value: ");
    scanf("%d", &N);
    printf("Maximum value: ");
    scanf("%d", &M);

    int numbers[X];
    generateNumbers(numbers, N, M, X);
    listNumbers(numbers, X);
    return 0;
}
```

SAMPLE RUN (random: your numbers will differ)

```
How many random numbers? 5
Minimum value: 10
Maximum value: 20
15
14
13
14
11
```

Note. The original class code used `N + rand() % (M + 1)` and was marked "formula is wrong". That formula can produce values up to N + M. The correct formula is `N + rand() % (M - N + 1)`. Printing is also kept out of the generating function, as the class notes recommended.

EXAMPLE 13

Practice: 10 different two-digit random numbers

Generate 10 random two-digit numbers (10-99) with no repeats. Before storing a new number, check that it is not already in the array.

CODE

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    int numbers[10];
    int count = 0;
    srand(time(NULL));

    while (count < 10) {
        int candidate = 10 + rand() % 90;   /* 10 ... 99 */
        int exists = 0;

        for (int i = 0; i < count; i++) {
            if (numbers[i] == candidate) {
                exists = 1;
                break;
            }
        }
        if (!exists) {
            numbers[count++] = candidate;
        }
    }

    for (int i = 0; i < 10; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");
    return 0;
}
```

SAMPLE RUN (random: your numbers will differ)

```
93 85 18 38 50 96 51 70 90 83
```

EXAMPLE 14

2D array: printing a matrix

Define a 2x3 integer matrix and print it row by row. The outer loop walks the rows and the inner loop walks the columns.

CODE

```c
#include <stdio.h>

int main() {
    int mat[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    printf("Matrix elements:\n");
    for (int i = 0; i < 2; i++) {           /* rows    */
        for (int j = 0; j < 3; j++) {       /* columns */
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

SAMPLE RUN

```
Matrix elements:
1 2 3
4 5 6
```

EXAMPLE 15

Character matrix

Define a 3x3 character matrix and print it.

CODE

```c
#include <stdio.h>

int main() {
    char mat[3][3] = {
        {'A', 'B', 'C'},
        {'D', 'E', 'F'},
        {'G', 'H', 'I'}
    };

    printf("Character matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%c ", mat[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

SAMPLE RUN

```
Character matrix:
A B C
D E F
G H I
```

EXAMPLE 16

Matrix addition with functions

Add two 2x2 matrices. One function adds them into a result matrix and another function prints a matrix.

CODE

```c
#include <stdio.h>

void addMatrices(int a[2][2], int b[2][2], int result[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

void displayMatrix(int mat[2][2]) {
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%d ", mat[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int mat1[2][2] = {{1, 2}, {3, 4}};
    int mat2[2][2] = {{5, 6}, {7, 8}};
    int result[2][2];

    addMatrices(mat1, mat2, result);

    printf("Result of matrix addition:\n");
    displayMatrix(result);
    return 0;
}
```

SAMPLE RUN

```
Result of matrix addition:
6 8
10 12
```

EXAMPLE 17

Encrypting a character matrix (Caesar shift)

Start from the 2x2 matrix {'X', 'Y'}, {'Z', 'A'}. Add a shift of 3 to the ASCII value of every letter to build an encrypted matrix, and print it row by row.

CODE

```c
#include <stdio.h>

int main() {
    char matrix[2][2] = {{'X', 'Y'}, {'Z', 'A'}};
    char encrypted[2][2];
    int shift = 3;

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            /* wrap around: X -> A, Y -> B, Z -> C */
            encrypted[i][j] = 'A' + (matrix[i][j] - 'A' + shift) % 26;
            printf("%c ", encrypted[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

SAMPLE RUN

```
A B
C D
```

Note. Adding the shift directly (`matrix[i][j] + shift`) turns X, Y and Z into the symbols [, \ and], because those come after Z in the ASCII table. The `% 26` version wraps around to the start of the alphabet.